SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1723**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Daniel Jesuraji (192565085) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

1. Introduction

The Water Jug Problem is a classical Artificial Intelligence problem used to demonstrate state-space search and problem-solving techniques.

In this problem, two unmarked jugs are available: one with a capacity of 4 gallons and another with a capacity of 3 gallons. A water pump is used to fill the jugs. Water can be poured from one jug to another or emptied onto the ground.

The objective is to obtain exactly 2 gallons of water in the 4-gallon jug without using any additional measuring devices.

2. Problem Representation

Let the state be represented as:

$$(x,y)$$

Where:

- x = amount of water in the 4-gallon jug.
- y = amount of water in the 3-gallon jug.

The initial state is:

$$(0,0)$$

The goal state is:

$$(2,y)$$

where y can be any amount from 0 to 3 gallons.

The available operations are:

1. Fill the 4-gallon jug.
2. Fill the 3-gallon jug.
3. Empty the 4-gallon jug.
4. Empty the 3-gallon jug.
5. Pour water from the 4-gallon jug into the 3-gallon jug.
6. Pour water from the 3-gallon jug into the 4-gallon jug.

3. Step-by-Step Solution

Initially, both jugs are empty.

Step	Operation	4-Gallon Jug	3-Gallon Jug
0	Initial state	0	0

1	Fill the 3-gallon jug	0	3
2	Pour from the 3-gallon jug into the 4-gallon jug	3	0
3	Fill the 3-gallon jug again	3	3
4	Pour from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full	4	2
5	Empty the 4-gallon jug	0	2
6	Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug	2	0

4. Verification of the Final State

The final state is:

\[\boxed{(2,0)} \]

Therefore, the 4-gallon jug contains exactly 2 gallons of water, and the 3-gallon jug is empty.

All operations follow the given assumptions, and no additional measuring equipment is required.

5. Conclusion

The Water Jug Problem can be solved using state-space search by representing each possible water quantity as a state and each filling, emptying or pouring operation as an action. By applying these actions systematically, the goal state \((2,0)\) is reached successfully. This problem demonstrates how AI can solve practical tasks through logical reasoning and a sequence of valid operations.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

1. Introduction

A Mars Rover is an autonomous robotic system designed to explore the surface of Mars, analyze soil and rock samples, collect scientific information and transmit data to Earth.

Since Mars is a distant planet, continuous human control is difficult because communication has significant delays. Therefore, the rover must use intelligent-agent techniques to perceive its surroundings, make decisions and perform scientific tasks autonomously.

(i) What Are the Percepts for This Agent?

Percepts are the information received by an intelligent agent from its environment through sensors.

The Mars Rover uses several sensors to understand the Martian environment.

1. Camera images: Cameras capture images of rocks, soil, terrain and nearby objects. These images help identify scientific targets and obstacles.
2. Distance and obstacle information: Sensors and cameras help estimate the distance to rocks, slopes and other objects that may block movement.
3. Soil and rock composition: Scientific instruments analyze the chemical and mineral composition of samples to understand the geological properties of Mars.

4. Temperature: Temperature sensors measure environmental conditions and help protect the rover's instruments.
5. Position and orientation: Navigation systems estimate the rover's location, direction and movement.
6. Power and system status: Battery levels, instrument health and other system measurements help the rover decide whether an action is safe and feasible.
7. Environmental conditions: Information about terrain, lighting and other environmental factors helps determine suitable exploration activities.
Thus, percepts provide the information required for navigation, scientific investigation and safe operation.

(ii) Characterize the Operating Environment

The Mars Rover operates in the following types of environments.
1. Partially observable: The rover cannot observe the entire Martian surface at once. It must use sensor data and stored maps to understand its surroundings.
2. Stochastic: Some outcomes are uncertain because terrain conditions, sensor readings and physical interactions may not be perfectly predictable.
3. Sequential: Each movement or scientific action affects future decisions and available opportunities.
4. Dynamic: Environmental conditions and the rover's internal state can change over time. The rover must respond to these changes.
5. Continuous: Physical variables such as position, distance, temperature and speed can take continuous values.
6. Single-agent: The rover is the primary autonomous decision-maker in the environment considered here, even though Earth-based operators may provide instructions.
7. Partly unknown: The rover may encounter terrain or geological features that were not fully known before exploration.
These characteristics require the rover to make decisions using incomplete information and update its knowledge as new observations become available.

(iii) What Are the Actions the Agent Can Take?

Actions are the operations performed by the agent to affect its environment or achieve its objectives.
The Mars Rover can perform the following actions:

1. Move: Travel forward, turn or reposition itself to explore the terrain.
2. Capture images: Take photographs of rocks, soil and surrounding areas.
3. Analyze samples: Use scientific instruments to examine the composition of rocks and soil.
4. Collect samples: Gather selected material for scientific investigation when equipped and instructed to do so.
5. Navigate: Select a safe route around obstacles and unsuitable terrain.
6. Transmit data: Send scientific results, images and status information to Earth.
7. Manage power: Schedule operations according to available energy.
8. Wait or enter a safe mode: Pause activities when conditions or system status require caution.

The rover selects actions based on its mission objectives, sensor information and operational limitations.

(iv) How Can the Performance of the Agent Be Evaluated?

The performance of the Mars Rover can be evaluated using a performance measure, which defines how successfully it achieves its objectives.
Important performance criteria include:
1. Scientific discoveries: The amount and quality of useful scientific information collected.
2. Navigation accuracy: The ability to reach selected destinations while avoiding obstacles.
3. Sample analysis quality: The accuracy and usefulness of measurements obtained from rocks and soil.
4. Energy efficiency: The ability to complete useful tasks while minimizing unnecessary power consumption.
5. Safety and reliability: The ability to protect the rover and its instruments from damage.
6. Mission completion: The extent to which the rover completes its assigned scientific and exploration objectives.
7. Communication effectiveness: The successful transmission of useful data to Earth.

A good rover agent maximizes scientific value and mission progress while maintaining safety and using limited resources efficiently.

(v) What Type of Agent Architecture Is Most Suitable? Why?

A model-based, goal-based agent with utility-based decision-making is suitable for a Mars Rover.

1. Model-Based Component

The rover maintains an internal representation of its location, surrounding terrain, discovered obstacles and previous observations. This is essential because the entire environment cannot be observed simultaneously.

2. Goal-Based Component

The rover selects actions that help achieve mission goals, such as reaching a scientific target, collecting a sample or investigating a geological formation.

3. Utility-Based Component

The rover compares possible actions according to their scientific value, energy requirements, safety and expected success. It can select a useful target without taking unnecessary risks.

Justification

A simple reflex agent would be insufficient for complex exploration because the rover needs to remember observations and plan future actions. A hybrid architecture allows it to maintain environmental knowledge, work toward mission goals and compare alternative actions.

For example, if two geological targets are available, the rover can consider their scientific importance, the energy required to reach them and the risks of the terrain before selecting one.

Conclusion

The Mars Rover demonstrates how intelligent agents can operate in partially observable and partly unknown environments. It receives percepts through sensors, performs navigation and scientific actions, and evaluates performance through mission progress, scientific discoveries, safety and efficiency. A hybrid model-based, goal-based and utility-based architecture is suitable because it supports informed planning and autonomous decision-making.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

1. Introduction

The 8-Queens Problem is a classical Artificial Intelligence problem in which eight queens must be placed on an 8×8 chessboard so that no queen attacks another queen.

In chess, a queen can attack another piece in the same row, column or diagonal. Therefore, the arrangement must ensure that no two queens share any of these positions.

The problem is commonly solved using state-space search and constraint satisfaction techniques.

2. Problem Formulation

A. Initial State

The initial state is an empty 8×8 chessboard with no queens placed.

B. State Representation

A state represents a partial arrangement of queens on the board.

We can represent the state as:

$$S=(r_1,r_2,\ldots,r_k)$$

Where:

- k is the number of queens placed.
- r_i is the row position of the queen in column i.

For example,

$$S=(2,4,1)$$

represents three queens placed in columns 1, 2 and 3, in rows 2, 4 and 1 respectively.
This representation places at most one queen in each column.

C. Initial State

$$S_0=()$$

The empty tuple indicates that no queens have been placed.

D. Actions

An action places a queen in a valid row of the next column.
For each column, the algorithm considers the available rows and rejects positions that would cause an attack with an already placed queen.

E. Transition Model

The transition model describes how the state changes when a queen is placed.
If the current state is:

$$S=(r_1,r_2,\ldots,r_k)$$

placing a queen in row r of the next column produces:

$$S'=(r_1,r_2,\ldots,r_k,r)$$

The new state is accepted only if it satisfies the constraints.

F. Goal Test

The goal state is reached when all eight queens have been placed without any two queens attacking one another.
Therefore,

$$\boxed{k=8}$$

and all row, column and diagonal constraints must be satisfied.

G. Path Cost

For a basic 8-Queens problem, each placement can be assigned a cost of 1. Since eight queens must be placed, every complete solution constructed this way requires eight placements.
The main objective is to find a valid arrangement rather than minimize a varying movement cost.

3. Constraints of the 8-Queens Problem

1. Row constraint: No two queens can occupy the same row.
2. Column constraint: No two queens can occupy the same column.
3. Diagonal constraint: No two queens can occupy the same diagonal.

If queen i is in row r_i, and queen j is in row r_j, their diagonal positions are valid only when:

$$|r_i-r_j|\ne|i-j|$$

This condition prevents two queens from sharing a diagonal.

4. Search Space Exploration

The search begins with an empty board and places one queen in each column.
The algorithm follows these steps:
Step 1: Start with an empty board.
Step 2: Place a queen in a valid row of column 1.
Step 3: Move to column 2 and select a row that does not conflict with the first queen.
Step 4: Continue placing queens in successive columns while checking the row and diagonal constraints.
Step 5: If no valid position exists in a column, backtrack by removing the previously placed queen and trying another row.
Step 6: Continue the process until all eight queens are placed successfully.
Step 7: If eight queens are placed without conflicts, the goal test succeeds.
This method is called backtracking search. It avoids exploring complete arrangements that already violate the constraints.

5. Example of a Valid Solution

One valid arrangement, represented by the row positions of queens in columns 1 through 8, is:

$$S=(1,5,8,6,3,7,2,4)$$

The queens are placed at the following coordinates:

$$\begin{aligned} &(1,1),(5,2),(8,3),(6,4),\\ &(3,5),(7,6),(2,7),(4,8) \end{aligned}$$

Each coordinate is written as (row, column).
The arrangement satisfies all three constraints:

- Every column contains exactly one queen.
- All eight queens occupy different rows.
- No two queens share a diagonal.

6. Search Strategy Used

Backtracking Search is suitable because it checks constraints whenever a queen is placed. If a conflict occurs, the algorithm returns to an earlier column and tries another position.

Depth-First Search can be used to explore the search tree, while constraint checking reduces unnecessary exploration.

For example, if placing a queen in column 4 creates a diagonal conflict, the algorithm rejects that placement immediately rather than continuing to place queens in columns 5 through 8.

Conclusion

The 8-Queens Problem is a constraint satisfaction problem that can be represented using states, actions, a transition model, a goal test and constraints. Backtracking Search explores possible arrangements systematically and rejects conflicting positions early. This approach efficiently finds a valid arrangement in which no two queens attack each other.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

1. Introduction

OLA is a mobile application that allows customers to book cabs to travel from one location to another. Customers can choose from different cab types, such as Mini, Micro, Sedan, Shared and Prime, depending on their comfort, budget and travel requirements.

A problem-solving agent can help the customer select a suitable cab and determine a route to the destination by considering the available options and constraints.

2. Type of Problem-Solving Agent

A goal-based problem-solving agent can be used to model the cab-booking problem. A utility-based decision component can additionally compare cab options based on price, comfort and estimated travel time.

Goal-Based Agent

A goal-based agent selects actions to achieve a specified goal.

In this problem, the main goal is:

To book a suitable cab and reach the destination successfully.

The agent follows these stages:

1. Identify the pickup location.
2. Identify the destination.
3. Obtain the available cab options.
4. Check the customer's preferences.
5. Select a suitable cab.
6. Confirm the booking.
7. Track the trip until the destination is reached.

Utility-Based Decision-Making

Different customers may have different priorities. One customer may prefer a low fare, while another may prefer greater comfort.

A utility-based component can evaluate:

- Estimated fare
- Estimated travel time
- Comfort level
- Cab availability
- Customer preference

It then selects the option that best satisfies the customer's priorities.

For example, a customer with a limited budget may prefer a Mini cab, while a customer prioritizing comfort may select a Sedan or Prime option if available.

3. Problem Formulation
Initial state: The customer has opened the cab-booking application.
Goal state: A suitable cab has been booked and the customer reaches the destination.
Actions: Enter pickup and destination, select a cab type, compare available options and confirm the booking.
Transition model: Each action updates the booking process, such as moving from entering locations to selecting a cab and then confirming the booking.
Goal test: The goal is achieved when the customer completes the trip at the specified destination.
Path cost: Depending on the customer's requirements, the cost can include the fare and estimated travel time. Comfort and availability may be included in the evaluation criteria.

4. Pseudocode for the Cab-Booking Agent

```
ALGORITHM CabBookingAgent

BEGIN

  INPUT pickup_location
  INPUT destination

  IF pickup_location is empty OR destination is empty THEN
    DISPLAY "Enter valid pickup and destination"
    STOP
  END IF

  available_cabs ← GetAvailableCabs(
    pickup_location,
    destination
  )

  IF available_cabs is empty THEN
    DISPLAY "No cabs available"
    STOP
  END IF

  INPUT customer_preference
  best_cab ← NONE
  best_utility ← -INFINITY

  FOR each cab IN available_cabs DO

    IF cab is available THEN

      utility ← Evaluate(
        cab.fare,
        cab.estimated_time,
        cab.comfort,
        customer_preference
      )

      IF utility > best_utility THEN
        best_utility ← utility
        best_cab ← cab
      END IF

    END IF

  END FOR
```

```
    IF best_cab is NONE THEN
       DISPLAY "No suitable cab found"
       STOP
    END IF

    DISPLAY best_cab details

    INPUT customer_confirmation

    IF customer_confirmation = YES THEN
       booking ← ConfirmBooking(best_cab)
       DISPLAY "Booking confirmed"
       TrackTrip(booking)
    ELSE
       DISPLAY "Booking cancelled"
    END IF

END
```

5. Explanation of the Pseudocode

Step 1: Input collection
The agent receives the pickup location and destination from the customer.
Step 2: Input validation
The agent checks whether both locations have been entered correctly.
Step 3: Find available cabs
The system obtains the available cab types for the selected route.
Step 4: Read customer preferences
The customer specifies preferences such as fare, comfort or travel time.
Step 5: Evaluate cab options
The agent evaluates the available cabs and calculates a utility score based on the customer's preferences.
Step 6: Select the best cab
The cab with the highest utility score is selected.
Step 7: Confirm the booking
The selected cab details are displayed, and the customer decides whether to confirm.
Step 8: Track the trip
After successful confirmation, the application tracks the trip until the customer reaches the destination.

6. Advantages

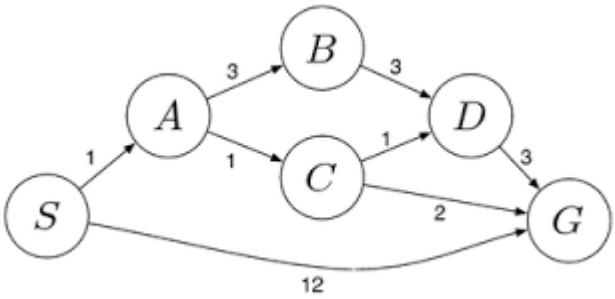
1. Helps customers compare different cab options.
2. Supports personalized decisions based on customer preferences.
3. Reduces the effort required to select a suitable cab.
4. Considers multiple factors, including fare, comfort and travel time.
5. Provides a structured decision-making process.

Conclusion

A goal-based problem-solving agent can manage the cab-booking process by working toward the goal of completing the customer's trip. Adding utility-based decision-making improves cab selection by comparing fare, comfort, availability and estimated travel time. This combination provides a flexible and systematic approach to intelligent cab booking.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



1. Introduction

Uniform Cost Search (UCS) is an uninformed search algorithm used to find the least-cost path from a starting node to a goal node in a weighted graph.

In a logistics company, warehouses are represented as nodes, and the routes connecting warehouses are represented as edges. Each edge has a travel cost based on factors such as fuel consumption and travel time.

The objective is to find the minimum-cost route from the starting warehouse \(S\) to the destination warehouse \(G\).

Important: The uploaded question paper refers to a weighted graph on the following page, but the actual graph and its edge weights are not available in the extracted question text. Therefore, the exact numerical path cannot be determined reliably from the provided information. The following explains the correct UCS procedure.

2. Problem Representation

The logistics network is represented as a weighted graph:

\[G=(V,E) \]

Where:

- \(V\) represents the warehouses.
- \(E\) represents the routes connecting the warehouses.
- \(c(u,v)\) represents the cost of travelling from warehouse \(u\) to warehouse \(v\).

The initial state is the starting warehouse \(S\).

The goal state is the destination warehouse \(G\).

The objective is to find a path \(P\) that minimizes the total cost:

\[C(P)=\sum_{(u,v)\in P}c(u,v) \]

3. Working Principle of UCS

UCS expands the node with the lowest accumulated path cost first.

The accumulated cost of a node \(n\) is represented by:

\[g(n)=g(\text{parent}(n))+c(\text{parent}(n),n) \]

The algorithm uses a priority queue in which the node with the smallest \(g(n)\) is selected first.

Unlike Breadth-First Search, UCS considers the cost of every route rather than only the number of edges.

4. Step-by-Step Algorithm

Step 1: Initialization

Insert the starting node \(S\) into the priority queue with cost zero.

\[Q=[(S,0)] \]

Step 2: Select the lowest-cost node

Remove the node with the smallest accumulated cost from the priority queue.

Step 3: Check the goal

If the selected node is \(G\), terminate the search. The current path is a least-cost path under the standard UCS assumptions.

Step 4: Expand the node

Generate all neighbouring warehouses connected to the current warehouse.

Step 5: Calculate path costs

For each neighbouring warehouse, calculate the new accumulated cost by adding the edge cost to the current path cost.

Step 6: Update the priority queue

If a lower-cost path to a warehouse is found, update its cost and parent information.
Step 7: Repeat
Continue expanding the lowest-cost node until the goal is reached or the priority queue becomes empty.
Step 8: Reconstruct the path
Use the stored parent information to reconstruct the route from \(S\) to \(G\), and calculate the total travel cost.

5. Pseudocode for UCS

```
ALGORITHM UniformCostSearch(Graph, S, G)

BEGIN

   PriorityQueue ← [(S, 0)]
   Cost[S] ← 0
   Parent[S] ← NONE

   WHILE PriorityQueue is not empty DO

      (Current, CurrentCost) ← RemoveMinimum(
         PriorityQueue
      )

      IF CurrentCost > Cost[Current] THEN
         CONTINUE
      END IF

      IF Current = G THEN
         RETURN ReconstructPath(Parent, G),
                Cost[G]
      END IF

      FOR each Neighbor of Current DO

         NewCost ← Cost[Current]
                   + EdgeCost(Current, Neighbor)

         IF Neighbor is not in Cost
           OR NewCost < Cost[Neighbor] THEN

            Cost[Neighbor] ← NewCost
            Parent[Neighbor] ← Current

            Insert(
               PriorityQueue,
               (Neighbor, NewCost)
            )

         END IF

      END FOR

   END WHILE

   RETURN "No path exists"

END
```

6. Illustration of Priority Queue Updates
The following is an illustrative example only, not the actual graph from the question paper.

Suppose the graph contains the edges:

- \(S\rightarrow A\), cost 2
- \(S\rightarrow B\), cost 5
- \(A\rightarrow G\), cost 6
- \(B\rightarrow G\), cost 1

Initially:
\[Q=[(S,0)] \]
After expanding \(S\):
\[Q=[(A,2),(B,5)] \]
The lowest-cost node is \(A\), so expand \(A\). The resulting path to \(G\) has cost:
\[2+6=8 \]
The queue contains the candidate \(B\) with cost 5 and \(G\) with cost 8.
Next, expand \(B\), whose cost is 5. The path through \(B\) reaches \(G\) with cost:
\[5+1=6 \]
Since 6 is less than 8, update the best-known cost of \(G\).
The queue now contains \(G\) with cost 6 and the older, more expensive entry for \(G\) with cost 8.
The lower-cost goal entry is selected first.
Therefore, the optimal path in this illustrative graph is:
\[\boxed{S\rightarrow B\rightarrow G} \]
Total cost:
\[\boxed{5+1=6} \]
This example demonstrates why UCS must update a node when a cheaper route is discovered.

7. Advantages of UCS

1. Finds the least-cost path when all edge costs are non-negative.
2. Handles weighted graphs effectively.
3. Considers the total travel cost instead of only the number of routes.
4. Can revise a node's best-known cost when a cheaper route is discovered.
5. Is complete when a solution is reachable and action costs have a positive minimum.

8. Conclusion

Uniform Cost Search is suitable for logistics networks because it identifies the least-cost route between warehouses by expanding nodes in increasing order of accumulated cost. The priority queue and cost-update mechanism allow the algorithm to compare alternative routes and select the cheapest one.
The exact final path and total cost for the assessment must be calculated from the original weighted graph and its edge values.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/for	Selects correct method	Inappropriate or partially correct method	Unable to select method

		mula with strong justification	with minor errors		
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem